DEEP LEARNING PROJECT

Crack Segmentation for Structural Monitoring

A U-Net that finds cracks pixel by pixel, and converts the result into a physical measurement a conservator can act on.

Team NeuraCoders

Tarrun B · Shyam M · Sriman A · Shyam Sundar K

MOTIVATION

Finding a crack is easy. Measuring one is not.

A structural engineer does not need to know that a wall has a crack — they can see that. They need to know whether it is wider than it was six months ago, and by how much. That is a measurement problem wearing a computer vision problem's clothes.

01 **Segment**

A network marks every pixel belonging to a crack. This is the deep learning part.

›

02 **Measure**

Skeletonise the mask, apply a distance transform, and recover width in pixels.

›

03 **Scale**

A surveyed reference in the photograph converts pixels to millimetres.

›

04 **Compare**

Two aligned captures months apart give the change that actually matters.

Segmentation alone is not measurement. A mask says which pixels. It does not say how wide, and it cannot say how much has changed. Steps 02 to 04 are what turn a vision model into an instrument.

THE CENTRAL DIFFICULTY

Cracks are 2% of the image

A model that predicts “no crack” everywhere scores roughly 98% pixel accuracy. The loss looks excellent and the model is useless. Almost every design decision in this project follows from that one fact.

LOSS ON A DO-NOTHING MODEL

Measured, not asserted — run python src/losses.py

Binary cross-entropy	0.31
Focal	0.23
BCE + Dice	0.65
Tversky	**0.99**
Dice	**0.99**
Focal Tversky	**0.99**

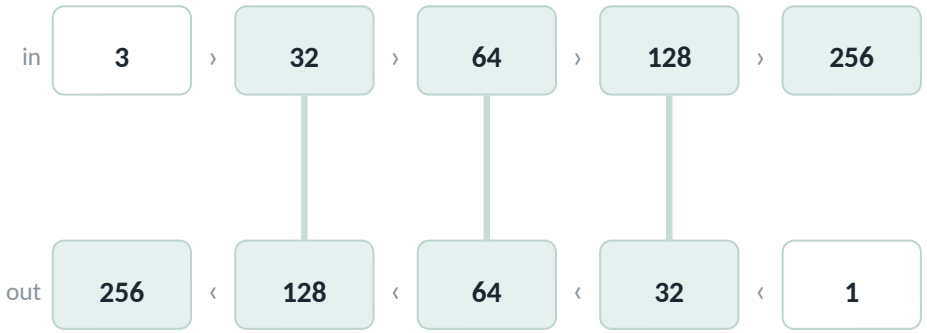
WHY IT MATTERS

- Cross-entropy barely penalises predicting nothing — 0.31 is a mediocre score, not a failing one.
- Overlap-based losses score the same model at 0.99, because an empty prediction has zero intersection.
- Tversky lets false negatives be punished harder than false positives, which is the right trade: a missed crack is a missed structural signal.
- Comparing all six on real data is the core experiment of this project.

METHOD

U-Net, written from scratch

An encoder compresses the image, a decoder expands it back to full resolution, and skip connections carry fine detail across. Thin structures need those skips — without them a one-pixel crack does not survive the bottleneck.



Vertical ties are skip connections. Channel widths at base 32, depth 4 — 7.8M parameters.

DESIGN DECISIONS

- Returns logits, never probabilities. Each loss applies its own sigmoid; a double sigmoid trains without error and quietly halves the learning signal.
- Trains on a free Colab GPU at base 32, so capacity is an ablation rather than a hardware constraint.
- Both upsampling modes implemented, so bilinear versus transposed convolution can be measured, not assumed.

Augmentation is constrained by the subject: flips and rotations are safe, but a scale change can erase a one-pixel crack while leaving its mask intact — teaching the model that blank wall means crack. Every geometric transform therefore uses nearest-neighbour interpolation on the mask, and the augmentation self-test checks that a thin crack survives every draw.

EVALUATION

Measuring it honestly

With 2% positive pixels, the choice of metric and the choice of protocol decide whether the numbers mean anything. Three decisions were made deliberately.

Metrics

IoU, Dice, precision and recall — reported per image and averaged, so a few heavily cracked images cannot dominate the score.

Pixel accuracy is reported only to show how misleading it is: the do-nothing model scores 0.98.

Threshold

The default 0.5 is rarely optimal on imbalanced data. A sweep selects it on the validation split, then it is applied once to test.

Tuning on test and reporting test is leakage, and it is an easy mistake to make by accident.

Splits

Train, validation and test are assigned by hashing the filename, not by shuffling with a seed.

An image lands in the same split every run, even after files are added. A re-shuffle that leaks test into train is very hard to spot from metrics alone.

THE DIFFERENTIATOR

From a mask to a millimetre

A segmentation network cannot measure crack width. Recovering a physical number takes three further steps, and each introduces error that has to be stated.

Skeletonise

Reduce the mask to a one-pixel centreline running along the crack.

Distance transform

At every crack pixel, the distance to the nearest background pixel.

Width

On the centreline, width = 2d − 1 pixels.

Scale

Multiply by mm-per-pixel from the surveyed reference.

CALIBRATION, MEASURED

true width	1	2	3	4	5	6	7
estimate	1	1	3	3	5	5	7

Odd widths are exact; even widths read one pixel short. Absolute widths therefore carry ±1 px of quantisation error.

WHAT THAT MEANS IN PRACTICE

At 0.5 mm per pixel, a reported “+2.0 mm” is really +2.0 ± 0.5 mm. Alert thresholds must sit above that floor. Stating the limit is worth more than hiding it — an engineer acting on these numbers needs to know when not to trust them.

RESULTS

Loss comparison — to be completed

The experiment: train the identical architecture six times, changing only the loss function, and report all four metrics. Fill this table from runs/*/test_metrics.json.

Loss	IoU	Dice	Precision	Recall
Binary cross-entropy	—	—	—	—
Dice	—	—	—	—
BCE + Dice	—	—	—	—
Focal	—	—	—	—
Tversky (β > α)	—	—	—	—
Focal Tversky	—	—	—	—

Expected finding. Cross-entropy under-segments and shows high precision with poor recall. Dice-family losses recover recall. Tversky with β > α should give the highest recall at some cost to precision — the right trade for structural monitoring, where a missed crack matters more than a false alarm.

LIMITATIONS

What this does not do

Four limits, stated rather than discovered by a reviewer. The first is the one that matters most.

Domain gap

Every public crack dataset is road and concrete. The target is stone masonry, where mortar joints are dark, linear and high-contrast — they look exactly like cracks. Expect the model to flag them. Road-surface metrics should not be presented as though they transfer.

Quantisation

Width carries ±1 px of error from the distance transform. Sub-pixel estimation needs grey-level profile fitting across the crack, which is out of scope here.

Scale dependency

Without a surveyed reference in the photograph, every output is in pixels and is not comparable between captures. The model cannot supply this; the capture protocol must.

Alignment assumption

Comparing two captures assumes they are spatially registered. Two hand-held photographs from different positions produce meaningless deltas.

ENGINEERING

Three workstreams, one contract

The project is split along two interfaces that are written down and machine-checked, so three people can build in parallel without an integration day spent on shape mismatches.

A Data

dataset, augmentation, class balance

`batches: float32 (N,3,H,W) + binary (N,1,H,W)`

›

B Model

U-Net, losses, training loop

`logits: raw (N,1,H,W), never probabilities`

›

C Measure

metrics, evaluation, measurement

`masks: uint8 (H,W) in {0,1}, plus statistics`

22 CONTRACT CHECKS, RUN BEFORE EVERY PUSH

Each workstream tests against stand-in implementations of the other two, so all three pass independently from day one — nobody waits. The checks catch the mistakes that actually break three-way splits: a mask that lost its channel dimension, a mask that stopped being binary after interpolation, a model returning probabilities instead of logits. If every section passes alone, the pipeline runs end to end.

CONCLUSION

Where this goes next

The segmentation model is a complete project in its own right. It was also built to be one component of a larger monitoring system.

DELIVERED

- U-Net implemented from scratch, both upsampling modes verified
- Six loss functions for the imbalance problem, comparable head to head
- Metrics that handle the empty-mask case rather than averaging NaNs
- Width measurement calibrated against known ground truth
- Machine-checked interfaces between three parallel workstreams

INTEGRATION TARGET

The wider system supplies the two things this model cannot: a scale reference, and two captures registered to the same control points.

```
seg = CrackSegmenter(ckpt)
m1  = seg.segment(march)
m2  = seg.segment(september)
delta = compare_captures(m1, m2,
                         mm_per_px=0.8)
```

A mask says which pixels. A measurement says whether to send an engineer.